

signal lab 1

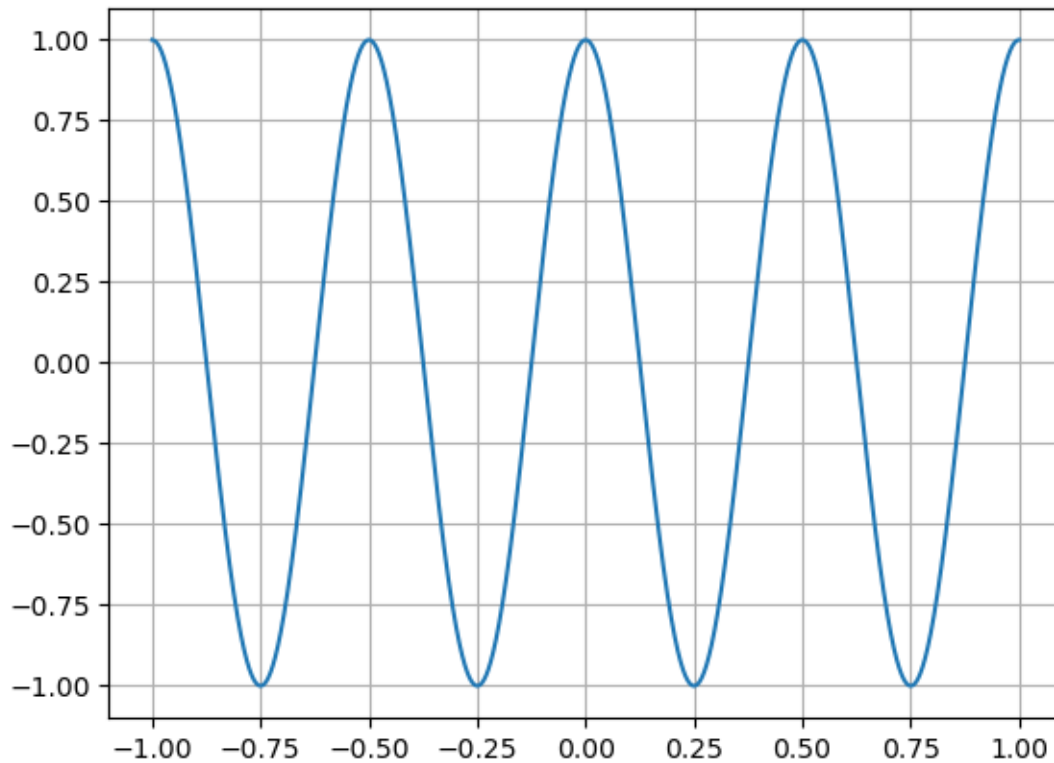
January 23, 2025

```
[2]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

fs = 44100 # 44,100-Hz sampling frequency
ts = 1/fs
t = np.arange(-1.,1.,ts) # A linearly-spaced array with step ts

#parameters and the function
A = 1.
f = 2 #2Hz
w0 = 2*np.pi*f
phi = 0.
xt = A*np.cos(w0*t + phi)

#plot
fig,ax = plt.subplots()
ax.plot(t,xt)
ax.grid(True)
plt.show()
```



```
[18]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

fs = 44100 # 44,100-Hz sampling frequency
ts = 1/fs
t = np.arange(-1.,1.,ts) # A linearly-spaced array with step ts
fig, axes = plt.subplots(2,2, sharex='all', sharey='all', figsize=(18,18))

#parameters and the function for xt1
A = 1.
f = 2 #2Hz
w0 = 2*np.pi*f
phi = 0.
xt1 = A*np.cos(w0*t + phi)

#parameters and the function for xt2
A = 0.5
f = 2 #2Hz
w0 = 2*np.pi*f
phi = 0.
xt2 = A*np.cos(w0*t + phi)
```

```

#parameters and the function for xt3
A = 1.
f = 2 #2Hz
w0 = 2*np.pi*f
phi = np.pi/2.
xt3 = A*np.cos(w0*t + phi)

#parameters and the function for xt4
A = 1.
f = 2 #2Hz
w0 = 4*np.pi*f
phi = 0.
xt4 = A*np.cos(w0*t + phi)

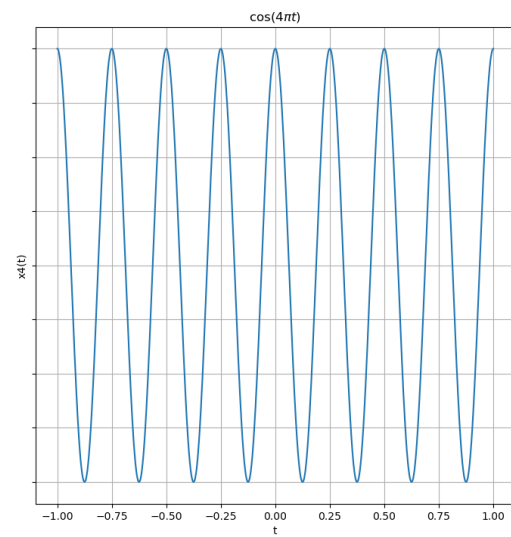
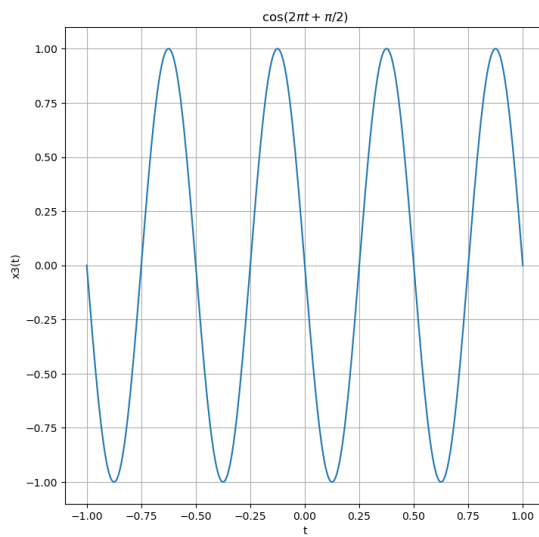
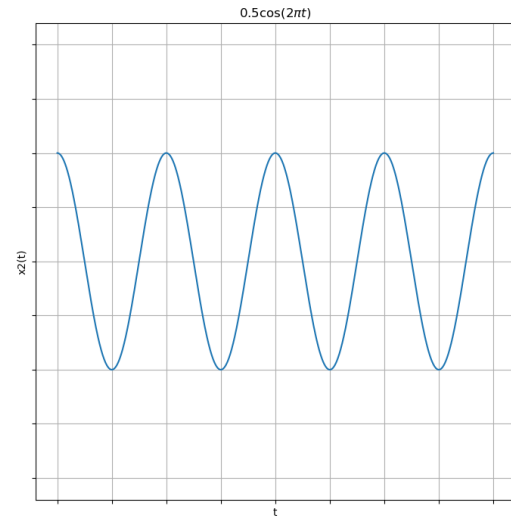
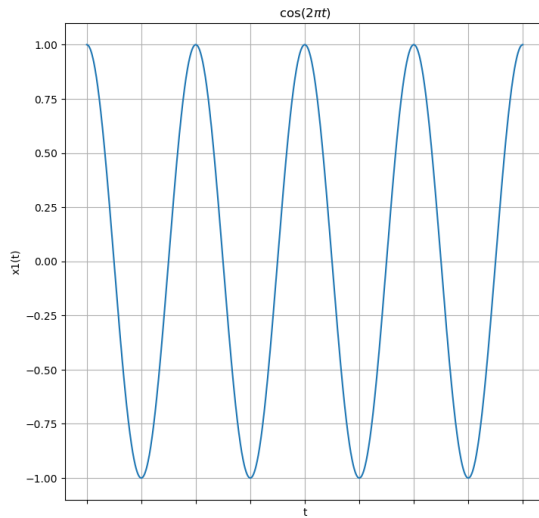
#plot
axes[0, 0].plot(t,xt1)
axes[0, 1].plot(t,xt2)
axes[1, 0].plot(t,xt3)
axes[1, 1].plot(t,xt4)

#labeling and titling
axes[0, 0].set_xlabel('t')
axes[0, 0].set_ylabel('x1(t)')
axes[0, 0].grid(True)
axes[0, 1].set_xlabel('t')
axes[0, 1].set_ylabel('x2(t)')
axes[0, 1].grid(True)
axes[1, 0].set_xlabel('t')
axes[1, 0].set_ylabel('x3(t)')
axes[1, 0].grid(True)
axes[1, 1].set_xlabel('t')
axes[1, 1].set_ylabel('x4(t)')
axes[1, 1].grid(True)

axes[0, 0].title.set_text('$\cos(2\pi t)$')
axes[0, 1].title.set_text('$0.5\cos(2\pi t)$')
axes[1, 0].title.set_text('$\cos(2\pi t + \pi/2)$')
axes[1, 1].title.set_text('$\cos(4\pi t)$')

plt.show()

```



```
[21]: fs = 4.4e4 # sampling frequency
      ts = 1/fs
      t = np.arange(-1., 1., ts)

      #common parameters for x1t and x2t
      f = 2 #2Hz
      w0=2*np.pi*f

      #unique parameters and the function for x1t
      phi = 0.
      x1t = np.cos(w0*t + phi)

      #unique parameters and the function x2t
```

```

phi = np.pi/2
x2t = np.cos(w0*t + phi)

#plot
fig, ax = plt.subplots()

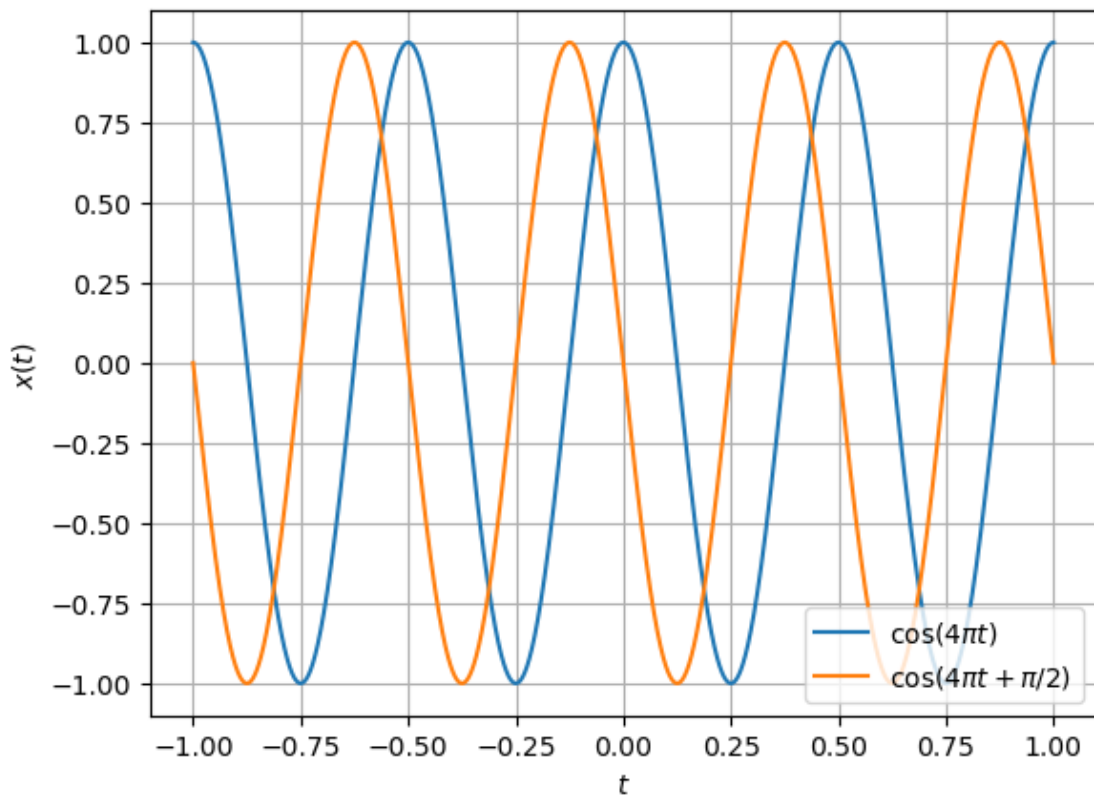
ax.plot(t,x1t, label='$\cos(4\pi t)$')
ax.plot(t,x2t, label='$\cos(4\pi t + \pi/2)$')

ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')

plt.legend(loc='lower right')

ax.grid(True)
plt.show()

```



```

[17]: fs = 4.4e4 # sampling frequency
      ts = 1/fs
      t = np.arange(-1., 1., ts)

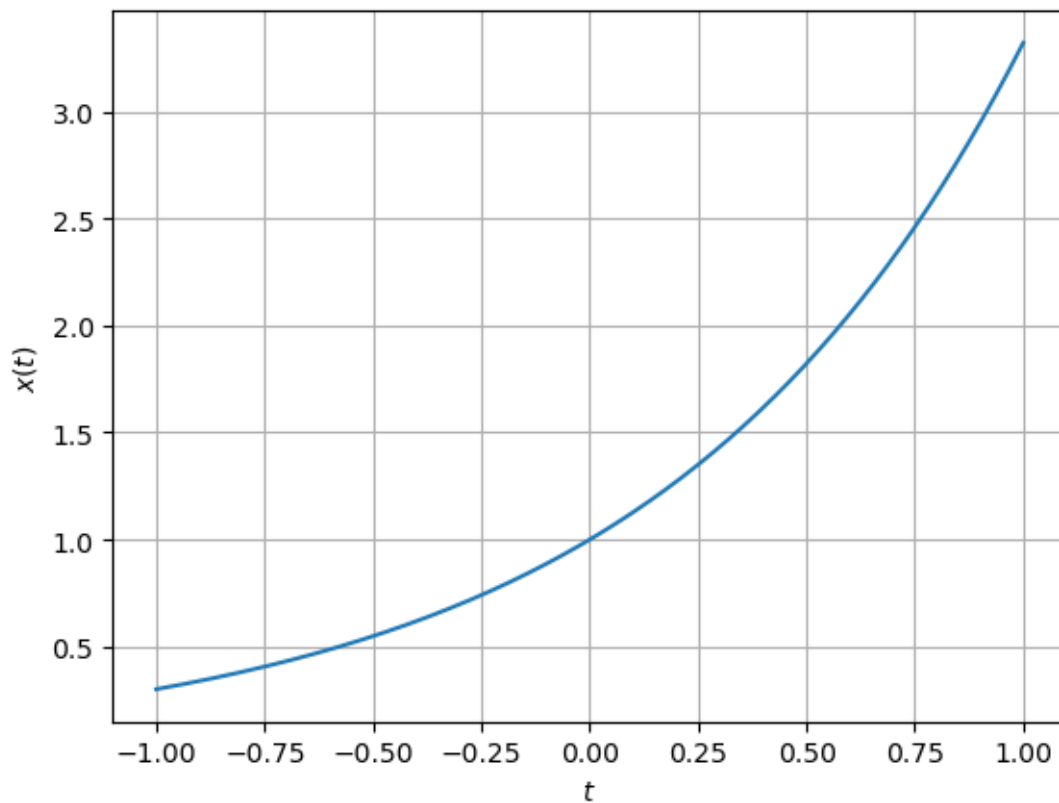
```

```

#parameters and the function
C = 1.0
alpha = 1.2
p = alpha*t
xt= C*np.exp(p)

#plot
fig,ax = plt.subplots()
ax.plot(t,xt)
ax.grid(True)
ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')
plt.show()

```



```

[3]: fs = 4.4e4 # sampling frequency
ts = 1/fs
t = np.arange(-1., 1., ts)

#parameters and the function
C = 1.0
alpha = -1.2

```

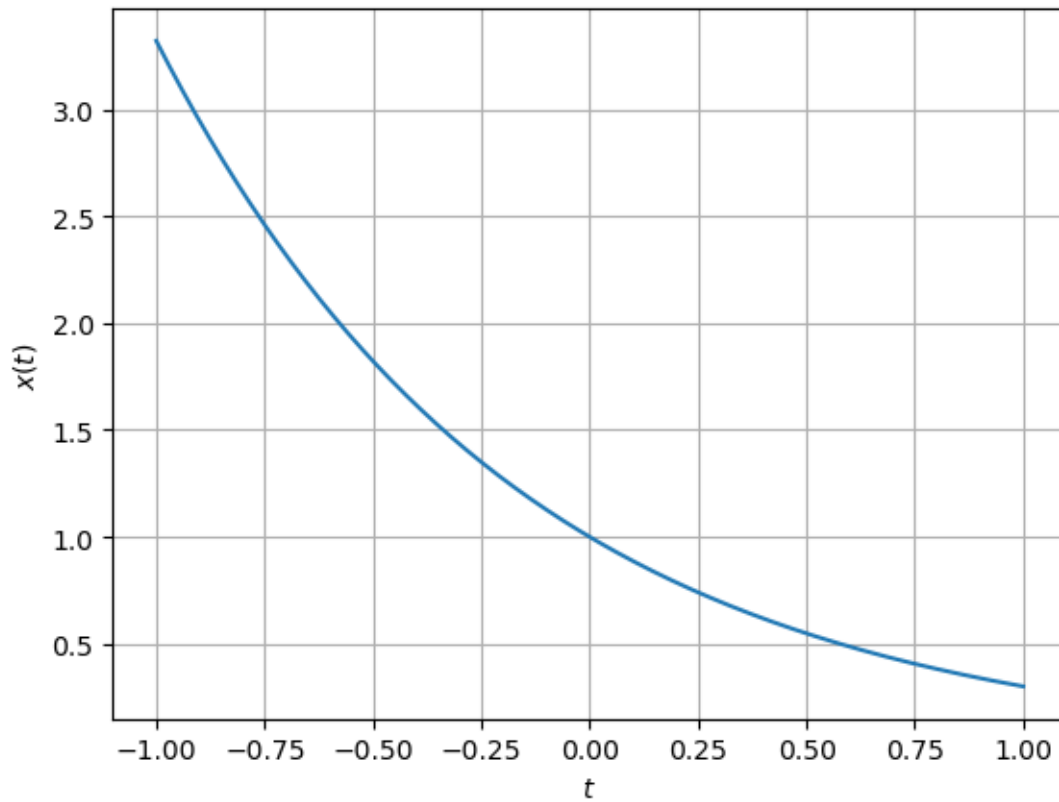
```

p = alpha*t
xt= C*np.exp(p)

#plot
fig,ax = plt.subplots()
ax.plot(t,xt)
ax.grid(True)

ax.set_xlabel('$t$')
ax.set_ylabel('$x(t)$')
plt.show()

```



```

[5]: fs = 4.4e4 # sampling frequency
ts = 1/fs
t = np.arange(-1., 1., ts)

#parameters and the function
C = 0.15
alpha = 2
p = alpha*t
f=5 #5Hz

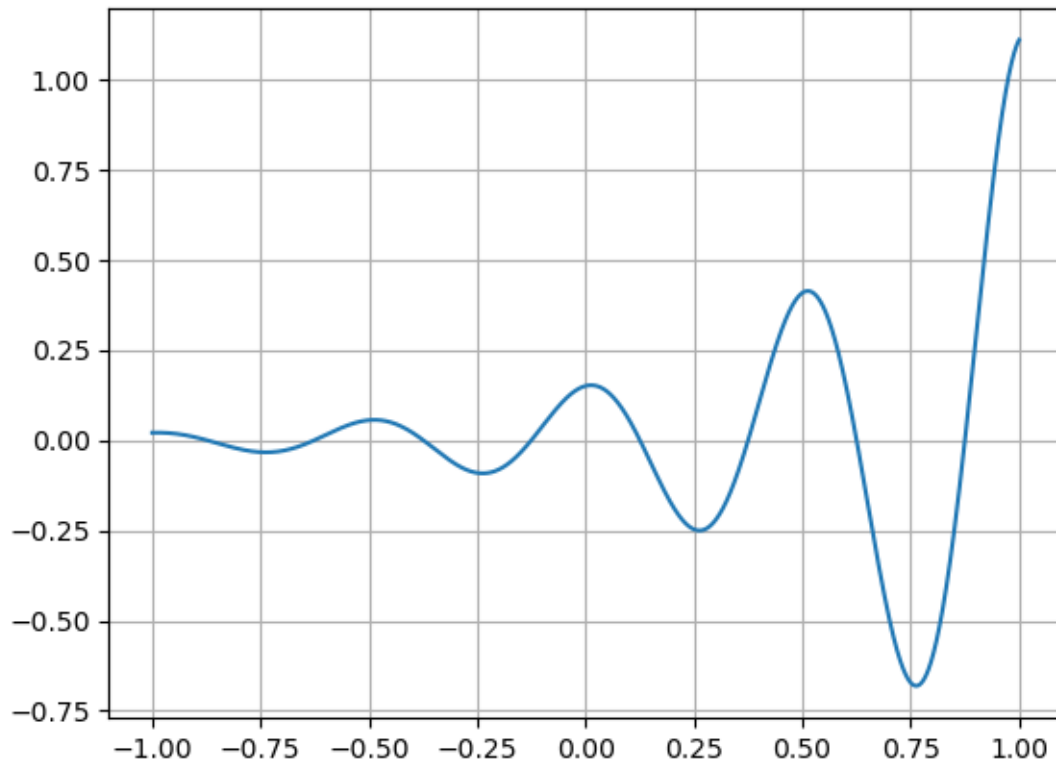
```

```

phi =0
wo =2*np.pi*f
xt= C*np.exp(p)*np.cos(w0*t + phi)

#plot
fig,ax = plt.subplots()
ax.plot(t,xt)
ax.grid(True)
plt.show()

```



```

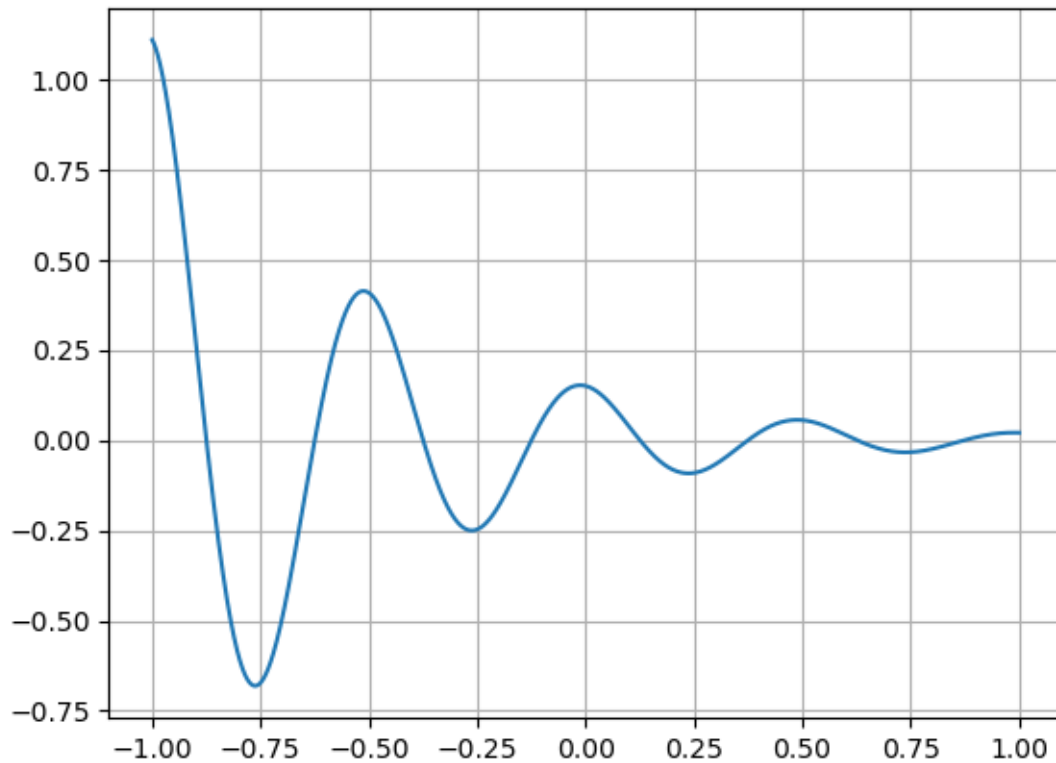
[6]: fs = 4.4e4 # sampling frequency
ts = 1/fs
t = np.arange(-1., 1., ts)

C = 0.15
alpha = -2
p = alpha*t
f=5
phi =0
wo =2*np.pi*f
xt= C*np.exp(p)*np.cos(w0*t + phi)

```



```
#plot
fig,ax = plt.subplots()
ax.plot(t,xt)
ax.grid(True)
plt.show()
```



```
[8]: n=np.arange(-10,10,1)
fig, axes = plt.subplots(2,2, sharex='all', sharey='all', figsize=(18,18))

#parameters and the function for xn1
C=1.0
alpha = 1.1
xn1 = C*alpha**(n)

#parameters and the function for xn2
C=1.0
alpha = 0.92
xn2 = C*alpha**(n)

#parameters and the function for xn3
C=1.0
alpha = -1.1
```

```

xn3 = C*alpha**(n)

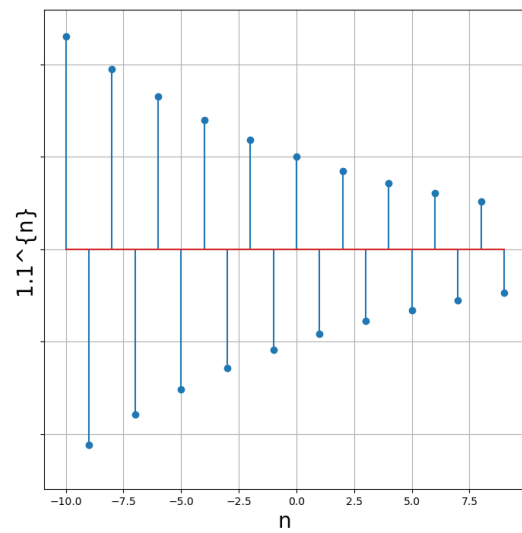
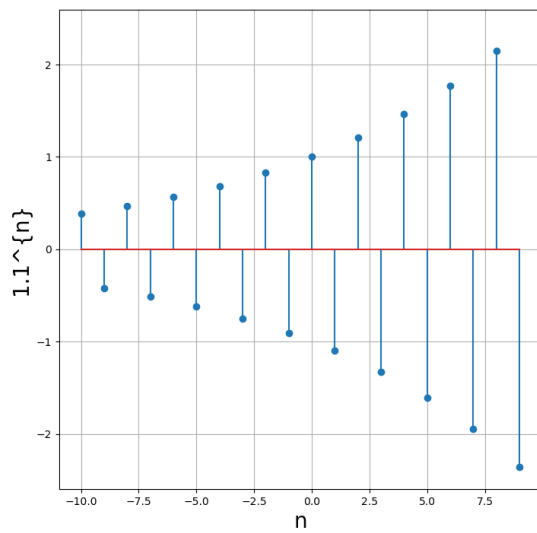
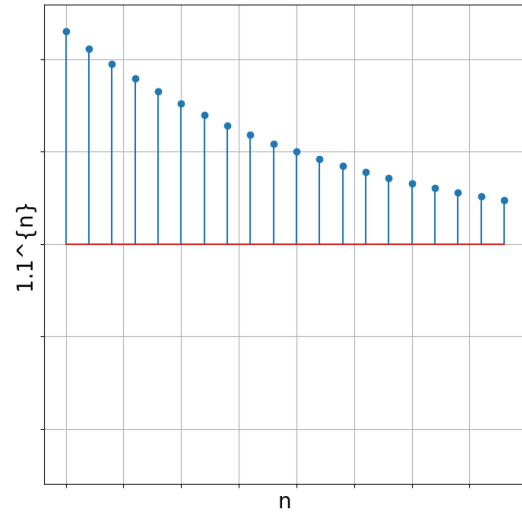
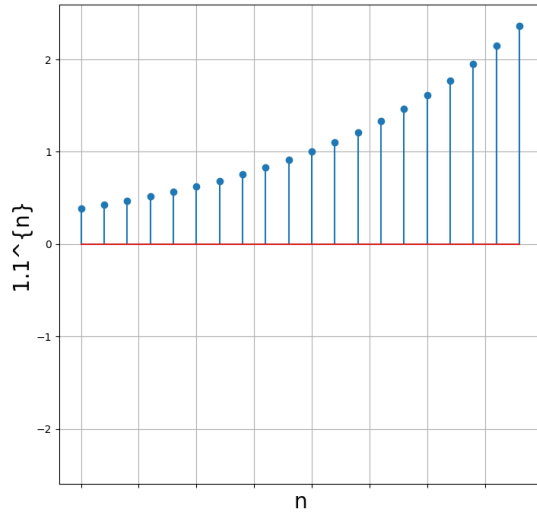
#parameters and the function for xn4
C=1.0
alpha = -0.92
xn4 = C*alpha**(n)

#plot
ax.plot(n,xn1 )
ax.plot(n,xn2 )
ax.plot(n,xn3 )
ax.plot(n,xn4 )

axes[0, 0].stem(n,xn1)
axes[0, 1].stem(n,xn2)
axes[1, 0].stem(n,xn3)
axes[1, 1].stem(n,xn4)
axes[0, 0].set_xlabel('n',fontsize =20)
axes[0, 0].set_ylabel('1.1{n}',fontsize =20)
axes[0, 0].grid(True)
axes[0, 1].set_xlabel('n',fontsize =20)
axes[0, 1].set_ylabel('1.1{n}',fontsize =20)
axes[0, 1].grid(True)
axes[1, 0].set_xlabel('n',fontsize =20)
axes[1, 0].set_ylabel('1.1{n}',fontsize =20)
axes[1, 0].grid(True)
axes[1, 1].set_xlabel('n',fontsize =20)
axes[1, 1].set_ylabel('1.1{n}',fontsize =20)
axes[1, 1].grid(True)

plt.show()

```



```
[9]: n = np.arange(-32,32,1)
fig, axes = plt.subplots(4,1, sharey='all', figsize=(18,18))

#parameters and the function for xn1
w0=2*np.pi/12
phi = 0.
xn1 = np.cos(w0*n + phi)

#parameters and the function for xn2
w0=2*np.pi/6
phi = 0.
xn2 = np.cos(w0*n + phi)
```

```

#parameters and the function for xn3
w0=8*np.pi/31
phi = 0.
xn3 = np.cos(w0*n + phi)

#parameters and the function for xn4
w0=1/1.5
phi = 0.
xn4 = np.cos(w0*n + phi)

#plot for xn1
ax.plot(n,xn1 )
axes[0].stem(n,xn1)
axes[0].set_xlabel('n' , fontsize =20)
axes[0].set_ylabel('$\cos(2\pi/12*n)$' , fontsize =20)
axes[0].grid(True)

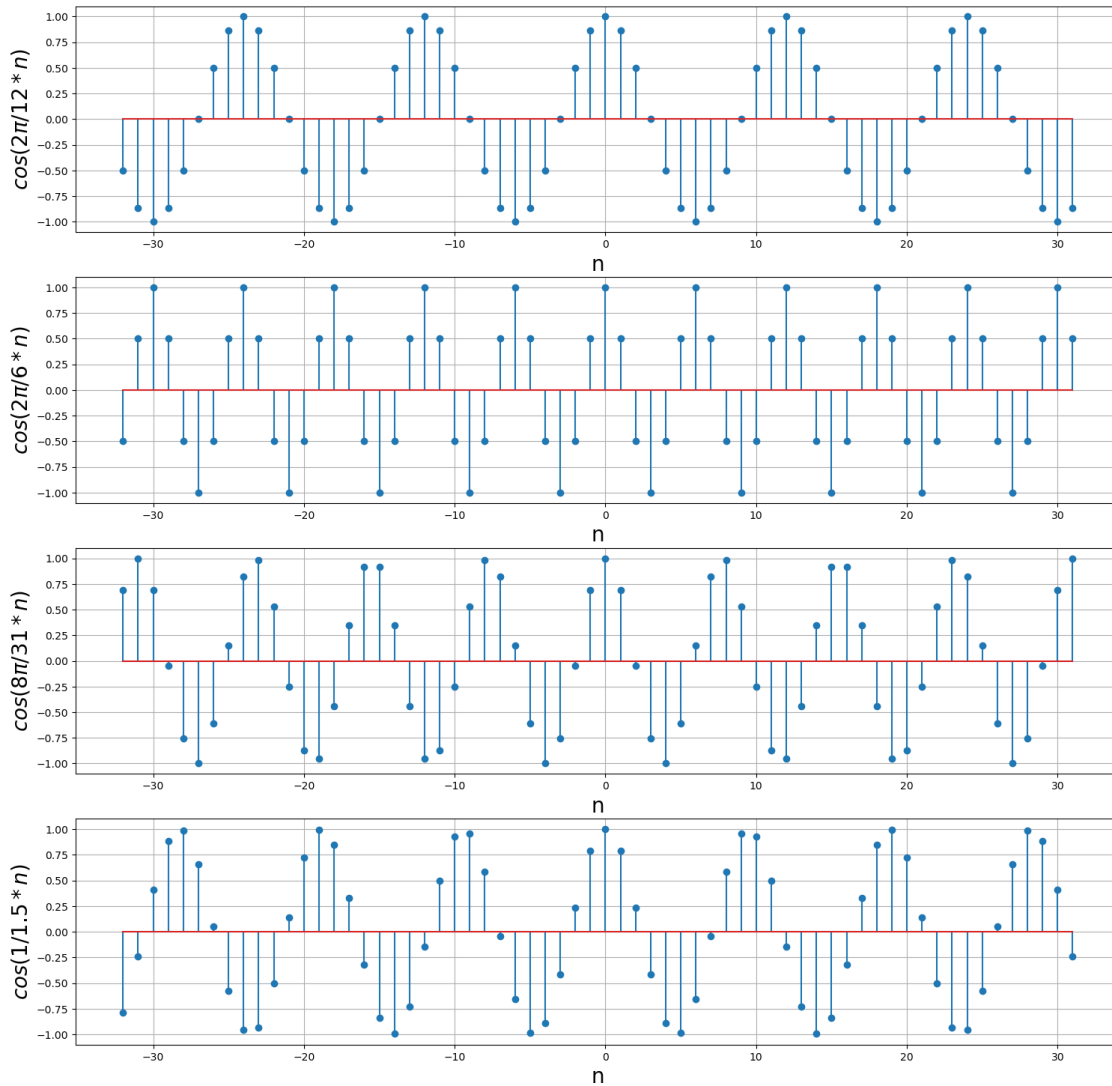
#plot for xn2
ax.plot(n,xn2 )
axes[1].stem(n,xn2)
axes[1].set_xlabel('n' , fontsize =20)
axes[1].set_ylabel('$\cos(2\pi/6*n)$' , fontsize =20)
axes[1].grid(True)

#plot for xn3
ax.plot(n,xn3 )
axes[2].stem(n,xn3)
axes[2].set_xlabel('n' , fontsize =20)
axes[2].set_ylabel('$\cos(8\pi/31*n)$' ,fontsize =20)
axes[2].grid(True)

#plot for xn4
ax.plot(n,xn4 )
axes[3].stem(n,xn4)
axes[3].set_xlabel('n',fontsize =20)
axes[3].set_ylabel('$\cos(1/1.5*n)$',fontsize =20)
axes[3].grid(True)

plt.show()

```



```
[11]: n = np.arange(-12,12,1)
fig, axes = plt.subplots(2,1, sharex='all', sharey='all', figsize=(18,18))
theta = 0.0

#parameters and the function for xn1
w0=2*np.pi/12
C = 1.
alpha = 1.1
xn1 = C*(alpha**n)*np.cos(w0*n + theta)

#parameters and the function for xn2
w0=2*np.pi/12
C = 1.
```

```

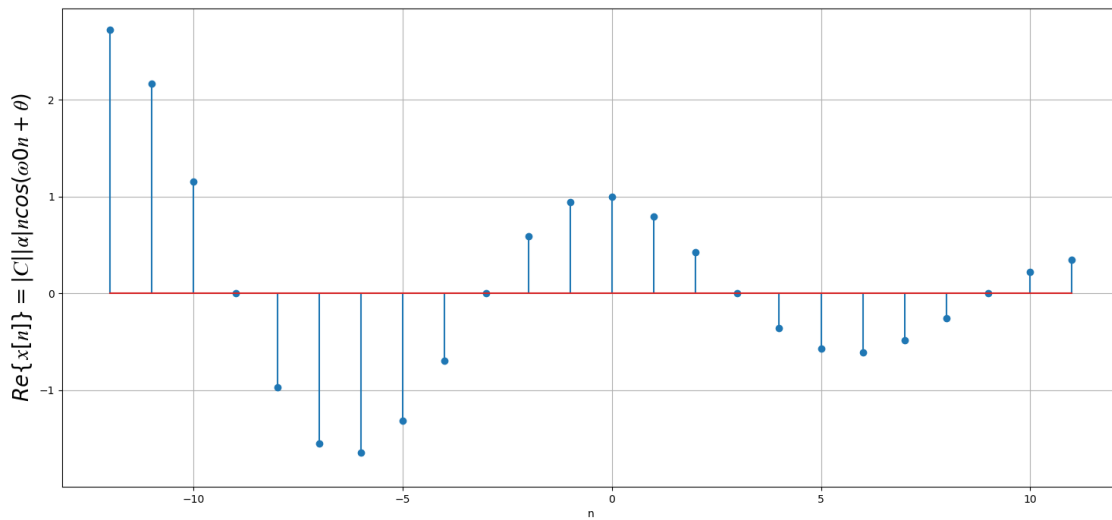
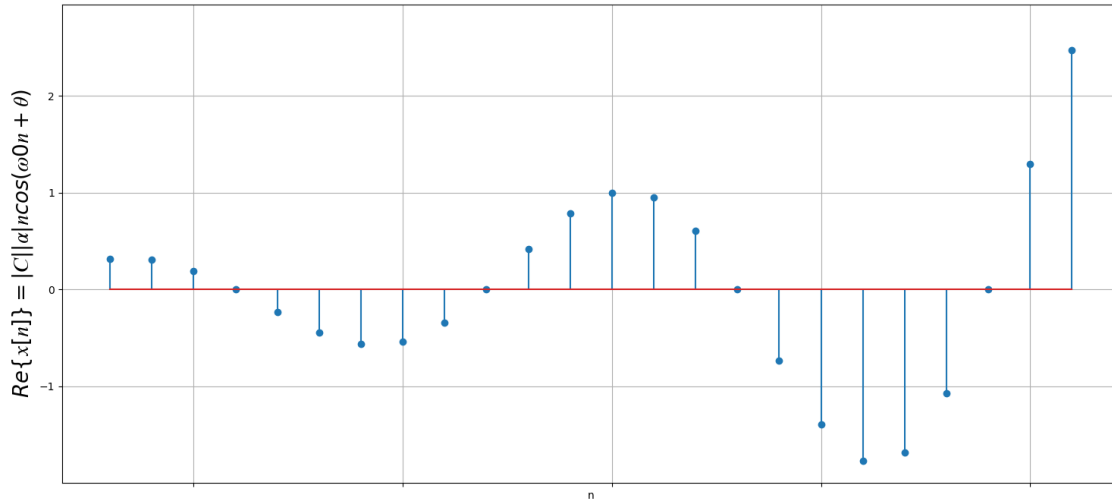
alpha = 0.92
xn2 = C*(alpha**n)*np.cos(w0*n + theta)

#plot
ax.plot(n,xn1 )
axes[0].stem(n,xn1)
axes[0].set_xlabel('n')
axes[0].set_ylabel('$Re\{ [ ] \} = ||| | \cos(0 + )$', fontsize =20)
axes[0].grid(True)

ax.plot(n,xn2 )
axes[1].stem(n,xn2)
axes[1].set_xlabel('n')
axes[1].set_ylabel('$Re\{ [ ] \} = ||| | \cos(0 + )$', fontsize =20)
axes[1].grid(True)

plt.show()

```



```
[34]: n = np.arange(-12,12,1)
fig, axes = plt.subplots(2,1, sharex='all', sharey='all', figsize=(18,18))
theta = 0.0

#parameters and the function for xn1
w0=2*np.pi/12
C = 1.
alpha = -1.1
xn1 = C*(alpha**n)*np.cos(w0*n + theta)

#parameters and the function for xn2
w0=2*np.pi/12
C = 1.
```

```

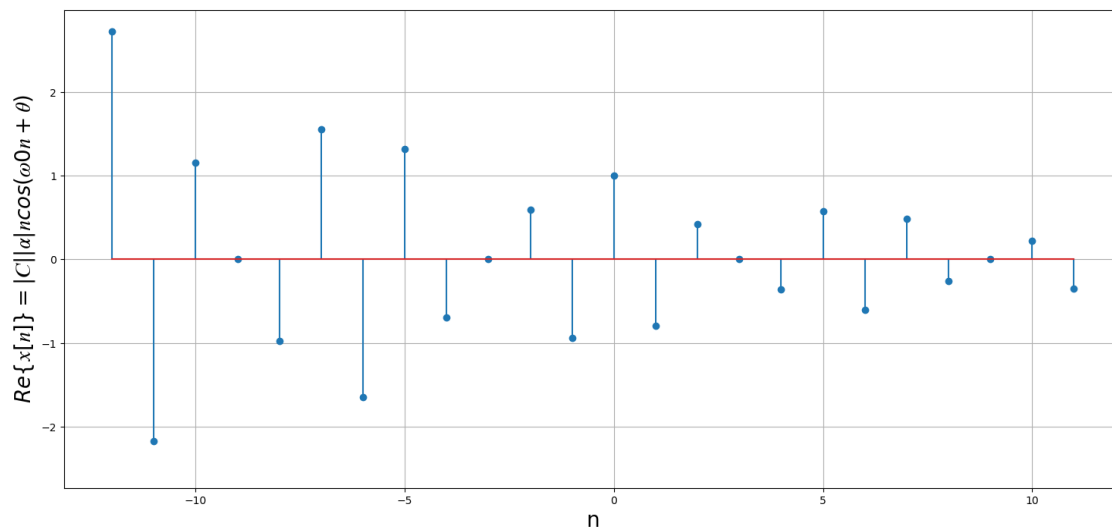
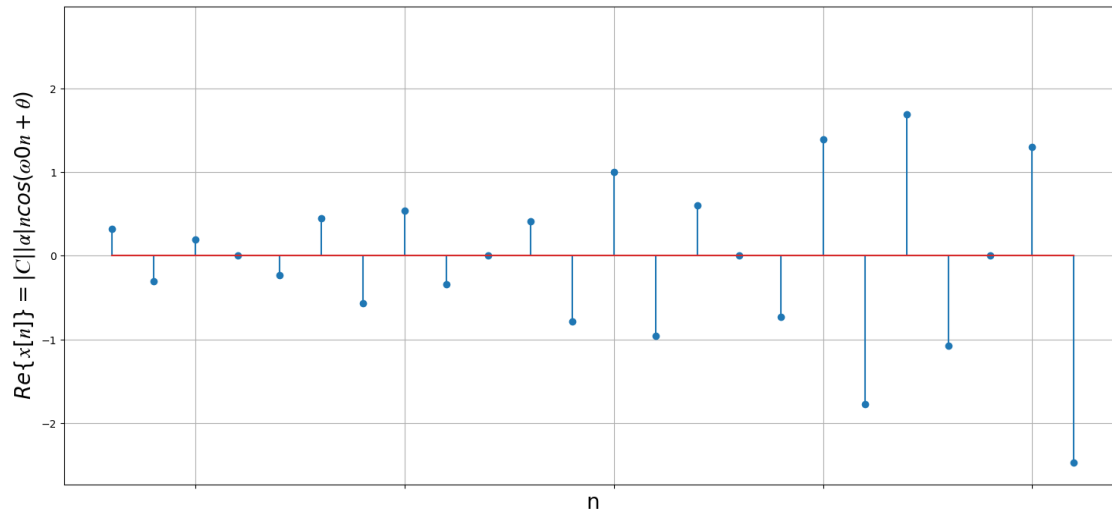
alpha = -0.92
xn2 = C*(alpha**n)*np.cos(w0*n + theta)

#plot
ax.plot(n,xn1 )
axes[0].stem(n,xn1)
axes[0].set_xlabel('n',fontsize =20 )
axes[0].set_ylabel('$Re\{ [ ] \} = ||| | \cos(0 + )$', fontsize =20)
axes[0].grid(True)

ax.plot(n,xn2 )
axes[1].stem(n,xn2)
axes[1].set_xlabel('n',fontsize =20)
axes[1].set_ylabel('$Re\{ [ ] \} = ||| | \cos(0 + )$', fontsize =20)
axes[1].grid(True)

plt.show()

```

[47]:

```
#defining the main function
def x(t):
    if (t < 0.):
        return 0.
    elif (t < 1.):
        return 1.
    elif (t < 2.):
        return 2. -t
    else:
        return 0.

fs = 4.4e4 # 44,000-Hz sampling frequency
```

```

ts = 1/fs
t = np.arange(-3.5, 3.5, ts) # A linearly-spaced array with step ts
fig, axes = plt.subplots(7,1, sharey='all', figsize=(10,15))

#  $x(t)$ 
axes[0].plot(t, [x(t_) for t_ in t])
axes[0].set_xlabel('$t$')
axes[0].set_ylabel('$x(t)$')
axes[0].grid(True)

#  $x(t-1)$ 
axes[1].plot(t, [x(t_-1) for t_ in t])
axes[1].set_xlabel('$t$')
axes[1].set_ylabel('$x(t-1)$')
axes[1].grid(True)

#  $x(t+1)$ 
axes[2].plot(t, [x(t_+1) for t_ in t])
axes[2].set_xlabel('$t$')
axes[2].set_ylabel('$x(t+1)$')
axes[2].grid(True)

#  $x(-t)$ 
axes[3].plot(t, [x(-t_) for t_ in t])
axes[3].set_xlabel('$t$')
axes[3].set_ylabel('$x(-t)$')
axes[3].grid(True)

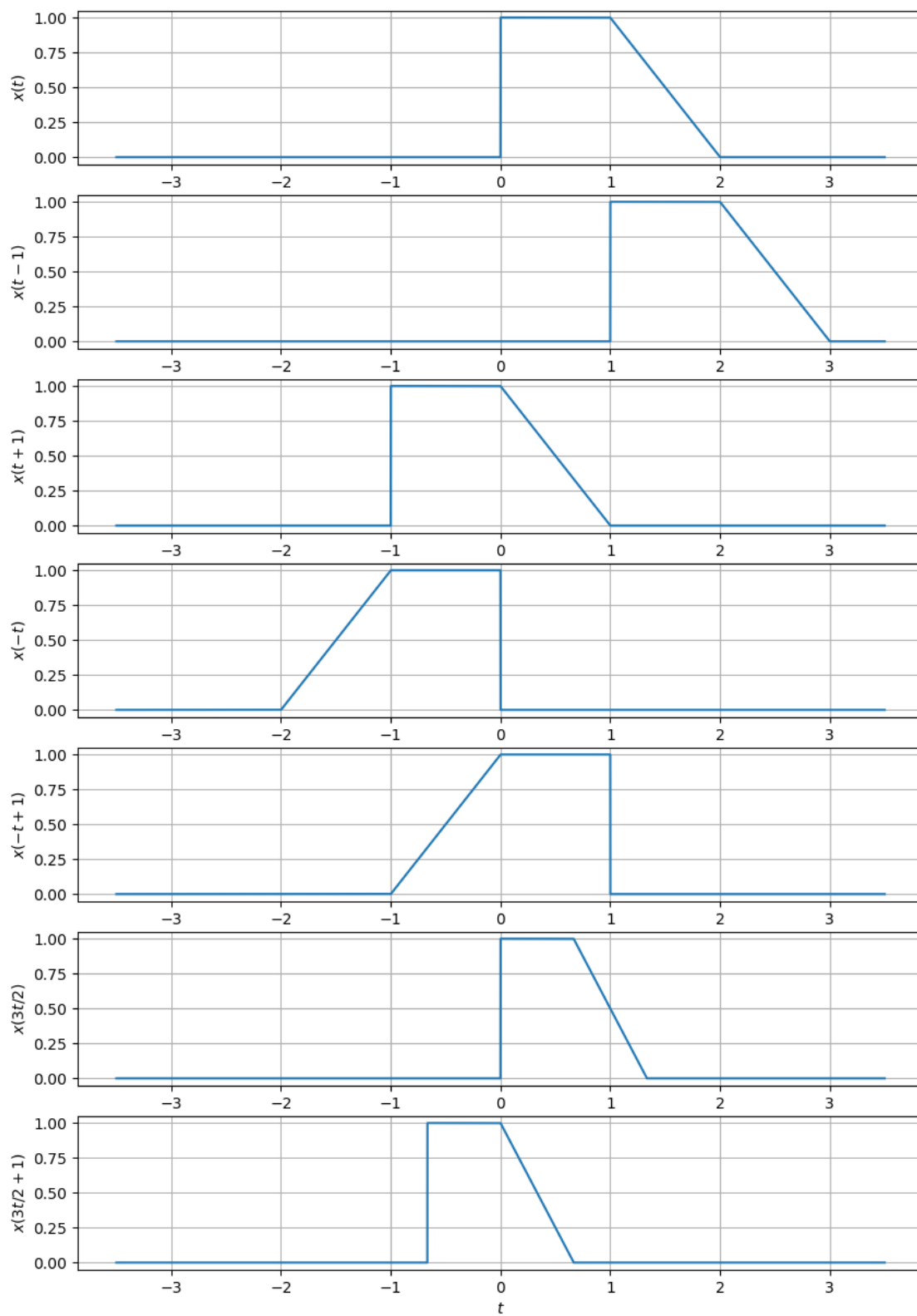
#  $x(-t+1)$ 
axes[4].plot(t, [x(-t_+1) for t_ in t])
axes[4].set_xlabel('$t$')
axes[4].set_ylabel('$x(-t+1)$')
axes[4].grid(True)

#  $x(3t/2)$ 
axes[5].plot(t, [x(1.5*t_) for t_ in t])
axes[5].set_xlabel('$t$')
axes[5].set_ylabel('$x(3t/2)$')
axes[5].grid(True)

#  $x(3t/2+1)$ 
axes[6].plot(t, [x(1.5*t_ +1) for t_ in t])
axes[6].set_xlabel('$t$')
axes[6].set_ylabel('$x(3t/2 +1)$')
axes[6].grid(True)

```

```
plt.show()
```



```

[12]: fs = 100# 100-Hz sampling frequency
      ts = 1/fs
      t = np.arange(-1., 1., ts) # A linearly-spaced array with step ts
      fig, axes = plt.subplots(2,1, figsize=(10,10))

      #signal in the time domain
      f = 2#2Hz
      omega0 = 2*np.pi*f
      xt = 0.75 + 2.*np.cos(omega0*t) + 1.*np.cos(3*omega0*t)

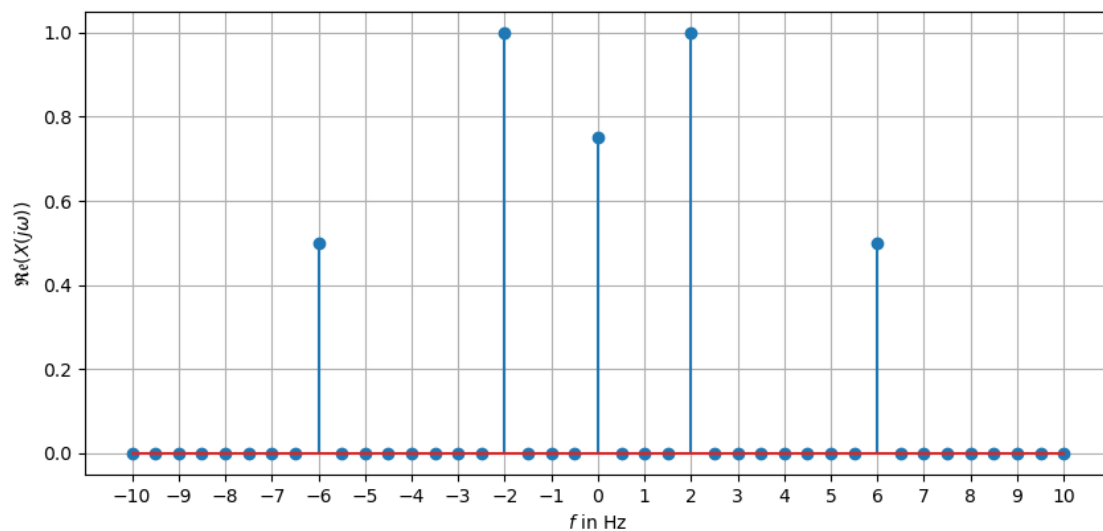
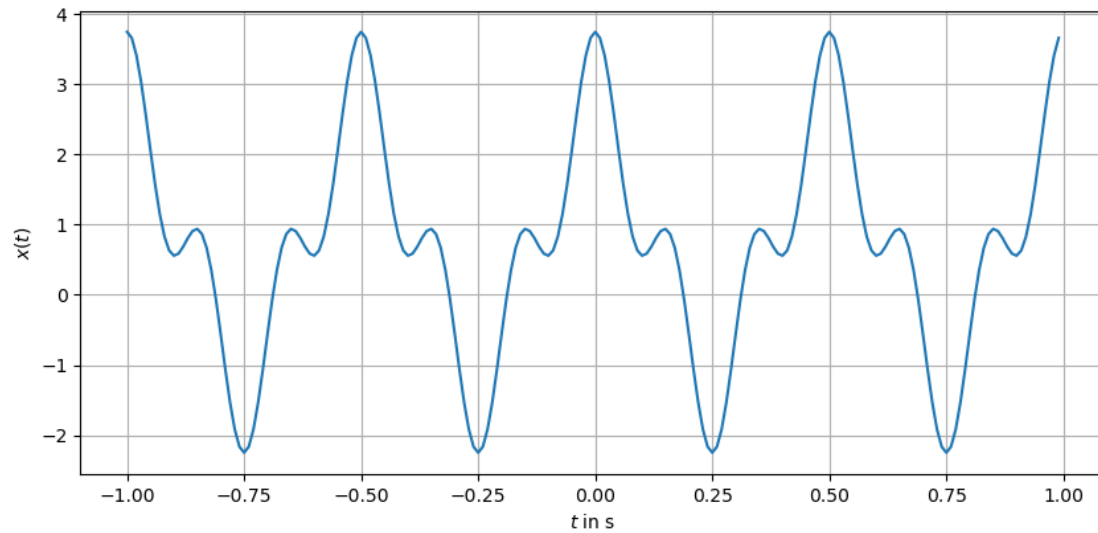
      axes[0].plot(t,xt)
      axes[0].set_xlabel('$t$ in s')
      axes[0].set_ylabel('$x(t)$')
      axes[0].grid(True)

      #signal in the frequency domain
      Xf = np.fft.fft(xt)
      freq = np.fft.fftfreq(t.shape[-1], d=ts)

      valsubrange = np.concatenate((np.arange(0,21,1), np.arange(-1,-21,-1)))
      freqsubrange = np.concatenate((np.arange(0,21,1), np.arange(-1,-21,-1)))
      axes[1].stem(freq[freqsubrange], Xf.real[valsubrange]/len(t))
      axes[1].set_xlabel('$f$ in Hz')
      axes[1].set_ylabel('$\mathfrak{Re}\{X(j\omega)\}$')
      axes[1].grid(True)

      plt.xticks(np.arange(-10,11))
      plt.show()

```



```
[16]: !pip install ffmpeg
```

```
!pip install pyaudio
```

Requirement already satisfied: ffmpeg in d:\anaconda\lib\site-packages (1.4)

Requirement already satisfied: pyaudio in d:\anaconda\lib\site-packages (0.2.14)

```
[5]: from IPython.display import Audio
# playing the echo wav file
Audio('anthem.wav')
```

```
[5]: <IPython.lib.display.Audio object>
```

```
[ ]:
```

```
[ ]: import numpy as np
import pyaudio
import wave
from IPython.display import Audio
    # import utility
CHUNK=8820#44100=1s
wf =wave.open('anthem.wav', 'r')
p =pyaudio.PyAudio()
nchannels=wf.getnchannels()
stream = np.array(np.zeros(nchannels), dtype=np.int16) # init stream
data = wf.readframes(CHUNK)
dtype = '<i2' # little-endian two-byte (int16) signed integers
sig = np.frombuffer(data, dtype=dtype).reshape(-1, nchannels)
signal_chunk = np.asarray(sig)
delayed = np.zeros(signal_chunk.shape, dtype=dtype)
i=0
alpha = 1.5
while data != '' and signal_chunk.shape[0] == CHUNK and i<120:
    i+=1
    modified_signal_chunk = alpha*signal_chunk + (3. -alpha)*delayed
    modified_signal_chunk_int16 = modified_signal_chunk.astype(np.int16)
    stream = np.vstack((stream, modified_signal_chunk_int16)) # append modified to
    ↪stream
    # byte_chunk = modified_signal_chunk_int16.tobytes()
    # stream.write(byte_chunk)
    delayed = signal_chunk
    data = wf.readframes(CHUNK)
    sig = np.frombuffer(data, dtype=dtype).reshape(-1, nchannels)
    signal_chunk = np.asarray(sig)
    stream = stream[1:] # pop stream init
    byte_stream = stream.tobytes() # np array to bytes
    p.terminate()
    wf.close()
    wfo =wave.open('anthem01.wav', 'wb') # writing the bot stream to a output wav
    ↪file
    wfo.setnchannels(nchannels)
    wfo.setsampwidth(wf.getsampwidth())
    wfo.setframerate(wf.getframerate())
    wfo.writeframes(byte_stream)
    wfo.close()
    Audio('anthem01.wav')
```

```
[ ]:
```